

No pasa nada con `salesPipeline.ts` ; déjalo creado y vacío por ahora. Es coherente con la arquitectura que vamos a documentar.

1. Crea la carpeta de documentación

En la raíz del proyecto, al mismo nivel que `src` , `public` y `package.json` , crea:

`docs`



Después crea este archivo:

`docs/PM_INTELLIGENCE_ARCHITECTURE_V1.md`



2. Pega este contenido completo

`</>` Markdown



```
# PM Intelligence — Architecture V1.0
```

```
## 1. Propósito del documento
```

Este documento define la arquitectura técnica oficial de PM Intelligence V1.

Su objetivo es mantener consistencia entre:

- Ingestión de datos.
- Validación.
- Normalización.
- Procesamiento.
- Persistencia.
- Motor de inteligencia.
- Presentación visual.

Toda nueva funcionalidad deberá respetar estas reglas antes de incorporarse al proyecto.

```
## 2. Visión del producto
```

PM Intelligence es una Web Application orientada a Product Managers y responsables comerc

La plataforma debe evolucionar desde un dashboard visual hacia un Business Operating Syst

- Recibir reportes operativos.
- Validar su estructura.
- Normalizar datos heterogéneos.
- Construir modelos comunes de negocio.
- Calcular indicadores.
- Detectar riesgos y oportunidades.

- Priorizar acciones.
- Alimentar un motor determinístico.
- Presentar información ejecutiva y operativa.
- Sustituir gradualmente datos demostrativos por datos reales.

3. Flujo general de datos

```text

Fuentes operativas

↓

Data Center

↓

Lectura y vista previa

↓

Validación

↓

Normalización

↓

Procesamiento

↓

Business Data Model

↓

Zustand Store

↓

Pulse Adapter

↓

Pulse Engine

↓

Pulse Intelligence Center

## 4. Principios arquitectónicos

### 4.1 Separación de responsabilidades

Cada capa debe tener una responsabilidad específica.

- Los componentes visuales presentan datos.
- Los validadores revisan estructura.
- Los normalizadores transforman valores.
- Los procesadores calculan resúmenes.
- Los stores conservan estado.
- Los adaptadores conectan dominios.
- El Pulse Engine evalúa reglas.
- Los widgets no contienen lógica de negocio.

### 4.2 Independencia de la fuente

El dashboard no debe depender de:

- Excel.
- CSV.
- Google Drive.
- NetSuite.
- APIs.
- Nombres concretos de columnas.

Cada fuente debe transformarse a un modelo normalizado antes de ser utilizada por el resto de la aplicación.

### 4.3 Datos reales y datos no disponibles

El sistema debe distinguir entre:

Valor igual a cero



y:

Dato no disponible



No se deben inventar métricas para dominios que no han sido cargados.

### 4.4 Procesamiento incremental

Los cambios deben implementarse en etapas pequeñas.

Antes de avanzar:

- TypeScript debe estar en cero errores.
- `npm run build` debe finalizar correctamente.
- Las rutas deben funcionar.
- La consola del navegador debe estar limpia.
- La funcionalidad anterior debe continuar operativa.

---

## 5. Stack tecnológico

### Frontend

- React.
- TypeScript.
- Vite.
- Tailwind CSS v4.
- HeroUI.

- React Router.
- Zustand.
- TanStack Query.
- Axios.
- Lucide React.

## Lectura de archivos

- SheetJS.
- Paquete `xlsx`.
- Procesamiento local en navegador durante la V1.
- Uso de `File.arrayBuffer()`.
- Uso de `XLSX.read()`.
- Uso de `XLSX.WorkBook`.
- Uso controlado de `XLSX.utils.sheet_to_json()`.

## 6. Estructura principal del proyecto

```
src/
├─ app/
├─ assets/
├─ atlas/
├─ engine/
├─ features/
├─ layouts/
└─ shared/
```



## 7. Estructura oficial del Data Center

```
src/features/data-center/
├─ adapters/
├─ components/
├─ importers/
│ ├─ sales/
│ ├─ inventory/
│ ├─ forecast/
│ ├─ quotas/
│ ├─ projects/
│ ├─ pricing/
│ └─ customers/
├─ mappings/
└─ normalizers/
```



```
├─ parsers/
├─ services/
├─ storage/
├─ store/
├─ types/
├─ utils/
├─ validators/
└─ DataCenterPage.tsx
```

## 8. Responsabilidad de cada carpeta

### adapters

Transforma resultados de un dominio hacia otro modelo.

Ejemplo:

```
SalesDatasetSummary
 ↓
Partial<PulseEngineInput>
```



Los adaptadores no deben recalcular el dataset completo.

### components

Contiene componentes visuales específicos del Data Center.

Ejemplos futuros:

- Selector de tipo de reporte.
- Zona de carga.
- Vista previa.
- Resumen de validación.
- Estado de procesamiento.
- Historial de importaciones.

### importers

Contiene los pipelines específicos de cada tipo de reporte.

Cada importador debe tener sus propios:

- Tipos.
- Aliases.
- Schema.
- Validador.
- Normalizador.

- Procesador.
- Pipeline.

#### mappings

Contiene mapas reutilizables entre estructuras externas y modelos internos.

#### normalizers

Contiene normalizadores compartidos entre distintos reportes.

Los normalizadores exclusivos de un reporte deben permanecer dentro de su importador.

#### parsers

Contiene lógica de lectura o interpretación de archivos.

Ejemplos:

- Excel.
- CSV.
- TSV.
- Detección de delimitadores.
- Detección de encabezados.

#### services

Contiene servicios de aplicación que coordinan operaciones.

No debe contener componentes React.

#### storage

Contiene mecanismos de persistencia.

Etapas previstas:

1. Memoria de sesión.
2. Zustand persist.
3. localStorage.
4. IndexedDB.
5. Backend.
6. Base de datos.

#### store

Contiene el estado global relacionado con el Data Center.

El store no debe almacenar datasets masivos.

Debe almacenar:

- Resúmenes.
- Metadatos.
- Estados.
- Errores.
- Cobertura de fuentes.
- Fechas de importación.

`types`

Contiene contratos compartidos.

Ejemplos:

- `ReportType` .
- `ImportStatus` .
- `ValidationResult` .
- `ImportResult` .
- `ImportPipeline` .
- `FileMetadata` .

`utils`

Contiene utilidades puras y reutilizables.

Ejemplos:

- Conversión de números.
- Conversión de monedas.
- Conversión de porcentajes.
- Conversión de fechas.
- Limpieza de cadenas.
- Preservación de identificadores.

`validators`

Contiene validadores compartidos.

Los validadores exclusivos de un reporte deben permanecer dentro de su importador.

---

## 9. Tipos de reporte admitidos

⌞ TypeScript



```
export type ReportType =
 | 'sales'
 | 'inventory'
 | 'forecast'
 | 'quota'
 | 'projects'
 | 'pricing'
 | 'customers'
 | 'other'
```

La arquitectura no debe quedar limitada al reporte de ventas.

Ventas será el primer pipeline operativo, no el único.

---

## 10. Contrato de importación

Todo importador debe respetar el mismo ciclo:

```
validate()
 ↓
normalize()
 ↓
process()
```



Contrato conceptual:

⌞ TypeScript



```
export interface ImportPipeline<
 TRawRow,
 TNormalizedRow,
 TSummary,
> {
 validate(headers: string[]): ValidationResult

 normalize(rows: TRawRow[]): TNormalizedRow[]

 process(rows: TNormalizedRow[]): TSummary
}
```

---

## 11. Pipeline por dominio

Cada dominio debe seguir esta estructura:

Raw File Rows



Domain Validator



Domain Normalizer



Domain Processor



Domain Summary



## Ejemplo para ventas:

Raw Sales Rows



salesValidator



salesNormalizer



salesProcessor



SalesDatasetSummary



## 12. Estructura oficial del Sales Importer

src/features/data-center/importers/sales/

```
├─ salesTypes.ts
├─ salesColumnAliases.ts
├─ salesSchema.ts
├─ salesValidator.ts
├─ salesNormalizer.ts
├─ salesProcessor.ts
└─ salesPipeline.ts
```



## Responsabilidades

salesTypes.ts

Define:

- NormalizedSalesRow .
- SalesByBrand .
- SalesByMonth .
- SalesByLocation .
- SalesDatasetSummary .

salesColumnAliases.ts

Define variaciones aceptadas de encabezados.

Ejemplo:

```
Transaction Total (Revenue)
Revenue
Venta
Ventas
Importe
```



```
salesSchema.ts
```

Define:

- Campos obligatorios.
- Campos recomendados.
- Campos opcionales.

```
salesValidator.ts
```

Debe:

- Reconocer alias.
- Crear el mapa de columnas.
- Detectar faltantes.
- Detectar columnas desconocidas.
- Determinar si el archivo puede procesarse.

```
salesNormalizer.ts
```

Debe convertir cada fila externa en `NormalizedSalesRow`.

No debe calcular indicadores agregados.

```
salesProcessor.ts
```

Debe calcular el resumen de ventas.

No debe conocer nombres de columnas del Excel.

```
salesPipeline.ts
```

Debe coordinar:

```
Validación
→ Normalización
→ Procesamiento
→ Resultado
```



No debe contener componentes visuales.

---

## 13. Reglas del Sales Importer

### Campos obligatorios

```
date
brand
revenue
```



Sin estos campos no debe permitirse el procesamiento.

### Campos recomendados

```
grossProfit
customerId
model
quantity
documentNumber
location
```



Su ausencia genera advertencias, pero no necesariamente bloquea el procesamiento.

### Campos opcionales

```
customerName
salesRep
currency
```



---

## 14. Modelo normalizado de ventas

📄 TypeScript



```
export interface NormalizedSalesRow {
 date: string
 brand: string
 revenue: number
 grossProfit: number
 customerId: string | null
 customerName: string | null
 model: string | null
 quantity: number
 documentNumber: string | null
 location: string | null
 salesRep: string | null
 currency: string | null
}
```

## 15. Resumen de ventas

🔗 TypeScript



```
export interface SalesDatasetSummary {
 periodStart: string | null
 periodEnd: string | null

 totalSales: number
 totalGrossProfit: number
 grossMargin: number
 totalQuantity: number

 uniqueCustomers: number
 uniqueProducts: number
 uniqueDocuments: number

 activeBrands: number
 activeLocations: number

 processedRows: number
 ignoredRows: number

 salesByBrand: SalesByBrand[]
 salesByMonth: SalesByMonth[]
 salesByLocation: SalesByLocation[]
}
```

## 16. Reglas de cálculo de ventas

### Margen bruto

Gross Margin = Total Gross Profit / Total Sales



Solo debe calcularse cuando `totalSales` sea distinto de cero.

### Facturas únicas

Se deduplican mediante:

Document Number



### Clientes únicos

Se deduplican mediante:

# cliente



## Productos únicos

Se deduplican mediante:

Modelo o SKU normalizado



## Marcas activas

Se deduplican mediante:

Marca



## Exclusiones

Se deben ignorar:

- Filas completamente vacías.
- Filas de totales.
- Valores nulos inválidos.
- Registros que no cumplan los mínimos definidos.
- Identificadores vacíos en conteos únicos.

---

## 17. Parsers compartidos

El archivo:

src/features/data-center/utils/valueParsers.ts



debe centralizar conversiones reutilizables.

Funciones iniciales:

📄 TypeScript



```
parseString()
parseNumber()
parseCurrency()
parsePercentage()
parseExcelDate()
parseIdentifier()
```

No deben duplicarse estas funciones dentro de cada importador.

## 18. Store del Data Center

Archivo:

```
src/features/data-center/store/dataCenterStore.ts
```



Debe permitir almacenar simultáneamente:

- `salesSummary`.
- `inventorySummary`.
- `forecastSummary`.
- `quotaSummary`.
- `projectsSummary`.
- `pricingSummary`.
- `customersSummary`.

Los dominios todavía no implementados pueden usar temporalmente `unknown`.

Ventas debe utilizar:

```
<> TypeScript
```



```
SalesDatasetSummary | null
```

El Store no debe borrar otros dominios al importar un nuevo reporte.

---

## 19. Estados de importación

```
<> TypeScript
```



```
export type ImportStatus =
 | 'idle'
 | 'reading'
 | 'preview'
 | 'validating'
 | 'processing'
 | 'completed'
 | 'error'
```

Flujo esperado:

```
idle
↓
reading
```



```
↓
preview
↓
validating
↓
processing
↓
completed
```

En caso de fallo:

```
error
```



## 20. Procesamiento de archivos

Formatos soportados:

```
.xlsx
.xls
.xlsm
.xlsb
.csv
.tsv
.ods
```



Reglas:

- No ejecutar macros.
- Conservar ceros a la izquierda.
- Aceptar fechas seriales, texto y objetos Date.
- Aceptar monedas y separadores.
- Detectar delimitadores CSV.
- Ignorar filas vacías.
- Detectar filas de totales.
- Informar columnas faltantes.
- Reportar filas ignoradas.

## 21. Rendimiento

La vista previa no debe procesar todo el archivo.

Debe almacenar únicamente:

- `previewRows`.

- `columns` .
- `totalRows` .
- `activeSheet` .
- `fileMetadata` .

El conteo de filas debe obtenerse mediante:

❏ TypeScript

```
worksheet['!ref']
```



y:

❏ TypeScript

```
XLSX.utils.decode_range()
```



La vista previa debe limitarse inicialmente a diez filas.

No se deben almacenar cientos de miles de filas dentro del estado de React ni Zustand.

---

## 22. Archivos grandes

El archivo mensual de aproximadamente 15,096 registros es adecuado para la primera versión.

Un archivo histórico de aproximadamente 885,000 registros requiere una estrategia diferente.

Opciones futuras:

- Web Worker.
- Procesamiento por lotes.
- Streaming para CSV.
- IndexedDB.
- Backend.
- Base de datos.

El navegador no debe considerarse un entorno ilimitado de procesamiento masivo.

---

## 23. Pulse Engine

El Pulse Engine es el punto central de evaluación de reglas.

⌞ TypeScript



```
const pulse = PulseEngine.evaluate(pulseEngineInput)
```

Debe alimentar:

- Executive Brief.
- Business Health.
- Critical Alerts.
- Recommendations.
- Opportunities.

Los widgets no deben evaluar reglas.

Los widgets no deben crear su propia variable `pulse`.

---

## 24. Adaptadores hacia Pulse

Cada dominio debe tener un adaptador.

Ejemplo:

```
src/features/data-center/adapters/salesToPulseAdapter.ts
```



Responsabilidad:

⌞ TypeScript



```
function salesSummaryToPulseInput(
 summary: SalesDatasetSummary,
) : Partial<PulseEngineInput>
```

El adaptador de ventas no debe inventar:

- Inventario.
- Forecast.
- Cuotas.
- Proyectos.
- Pricing.

---

## 25. Fuentes parciales

El `PulseEngineInput` debe evolucionar hacia dominios opcionales.

Ejemplo conceptual:

❏ TypeScript



```
type PulseEngineInput = {
 sales?: SalesDomainInput
 inventory?: InventoryDomainInput
 forecast?: ForecastDomainInput
 quotas?: QuotaDomainInput
 projects?: ProjectsDomainInput
 pricing?: PricingDomainInput
}
```

El motor debe evaluar únicamente las fuentes disponibles.

---

## 26. Reglas de componentes React

- No declarar variables `const` dentro del JSX.
- Preparar los datos antes del `return`.
- Declarar explícitamente los tipos de props.
- No instanciar un componente dentro de su propio archivo.
- No procesar filas crudas dentro de widgets.
- No calcular reglas dentro de tarjetas.
- No duplicar lógica entre componentes.

---

## 27. Reglas de TypeScript

Antes de avanzar:

❏ Bash



```
npm run build
```

debe finalizar sin errores.

Los archivos de respaldo no deben conservar extensión `.tsx` dentro de rutas analizadas.

Usar extensiones como:

`.bak`

`.txt`



Se deben eliminar:

- Imports no utilizados.
  - Variables antiguas.
  - Tipos redundantes.
  - Implementaciones duplicadas.
- 

## 28. Navegación

Ruta del Data Center:

```
<> TypeScript 

{
 path: 'data-center',
 element: <DataCenterPage />,
}
```

Elemento del Sidebar:

```
<> TypeScript 

{
 label: 'Data Center',
 path: '/data-center',
 icon: Database,
}
```

Debe utilizarse `path`, no `href`.

---

## 29. Regla de estabilidad

No realizar simultáneamente:

- Rediseño completo.
- Migración de motor.
- Cambio de estado global.
- Incorporación de varios importadores.
- Conexión con fuentes externas.

Cada etapa debe cerrar funcionalmente antes de comenzar la siguiente.

---

## 30. Orden oficial de implementación V1

1. Estructura del Data Center.

2. Tipos comunes.
  3. Store.
  4. Tipos de ventas.
  5. Aliases de ventas.
  6. Schema de ventas.
  7. Validador de ventas.
  8. Parsers compartidos.
  9. Contrato de pipelines.
  10. Normalizador de ventas.
  11. Procesador de ventas.
  12. Pipeline de ventas.
  13. Integración con DataCenterPage.
  14. Botón Procesar datos.
  15. Persistencia del resumen.
  16. Adaptador de ventas a Pulse.
  17. Sustitución de métricas ficticias.
  18. Cobertura de fuentes.
  19. Historial de importaciones.
  20. Importadores adicionales.
- 

## 31. Estado actual del proyecto

Completado:

- Estructura de carpetas.
- Ruta de Data Center.
- Navegación en Sidebar.
- Topbar dinámico.
- Tipos comunes.
- Store multidominio.
- Tipos de ventas.
- Aliases.
- Schema.
- Validador.
- Parsers compartidos.
- Contrato de pipelines.
- Archivo `salesPipeline.ts` creado.

Pendiente inmediato:

```
salesNormalizer.ts
```



Después:

```
salesProcessor.ts
```



y posteriormente:

```
salesPipeline.ts
```



---

## 32. Regla de gobierno técnico

Este documento es la referencia arquitectónica oficial de PM Intelligence V1.

Cuando una propuesta contradiga este documento deberá:

1. Justificarse técnicamente.
2. Evaluar impacto.
3. Actualizar la arquitectura.
4. Validarse antes de implementarse.

No debe modificarse la arquitectura de manera accidental durante el desarrollo.

```
3. Sobre `salesPipeline.ts`
```

```
La ruta correcta es:
```

```
```text
```

```
src/features/data-center/importers/sales/salesPipeline.ts
```



Por ahora debe permanecer vacío. No pegues lógica todavía.

4. Validación

Al ser un archivo Markdown dentro de `docs`, no afectará TypeScript ni la compilación. No es necesario ejecutar `npm run build` únicamente por este documento.

El siguiente archivo funcional será exactamente:

```
src/features/data-center/importers/sales/salesNormalizer.ts
```



Ahí comenzaremos la transformación real de las filas del archivo.

